Lab 12: Regularization, Trees, and Cross-Validation

SOCIOL 723 — Social Statistics II — Thursday, November 12

Wenhao Jiang

Table of contents

1 Goals

1. See the bias–variance tradeoff by simulating it.
2. Fit ridge, lasso, and elastic net, and choose λ by cross-validation.
3. Fit trees, random forests, and boosting, and compare honest test error.
4. Watch what happens when cross-validation is done wrong.
5. See why lasso's selected set is not a variable-importance ranking.

```
library(dplyr)
library(ggplot2)
library(glmnet)
library(ranger)
library(rpart); library(rpart.plot)

gss <- readRDS("../Data/gss_earnings.rds")
set.seed(723)
```

2 The tradeoff, simulated

```r
f_true <- function(x) sin(2 * x) + 0.5 * x

sim_once <- function(n = 60, df) {
  x <- runif(n, -3, 3); y <- f_true(x) + rnorm(n, sd = .6)
  fit <- smooth.spline(x, y, df = df)
  predict(fit, x = 1.0)$y            # predict at a fixed point
}

grid <- expand.grid(df = c(2, 4, 8, 16, 30))
res <- lapply(grid$df, function(d) replicate(400, sim_once(df = d)))

tibble::tibble(
  df = grid$df,
  bias2 = sapply(res, function(r) (mean(r) - f_true(1.0))^2),
  variance = sapply(res, var)
) |> mutate(mse = bias2 + variance) |> knitr::kable(digits = 4)
```

df	bias2	variance	mse
2	1.1528	0.0169	1.1697
4	0.4504	0.0238	0.4742
8	0.0093	0.0321	0.0414
16	0.0015	0.1070	0.1085
30	0.0007	0.2197	0.2204

```r
tibble::tibble(df = grid$df,
               bias2 = sapply(res, function(r) (mean(r) - f_true(1.0))^2),
               variance = sapply(res, var)) |>
  mutate(MSE = bias2 + variance) |>
  tidyr::pivot_longer(-df) |>
  ggplot(aes(df, value, colour = name)) +
  geom_line(linewidth = .9) + geom_point() +
  scale_colour_manual(values = c("bias2" = "navy", "variance" = "firebrick",
                                 "MSE" = "black")) +
  labs(x = "effective degrees of freedom", y = NULL, colour = NULL) +
  theme_minimal(base_size = 10)
```

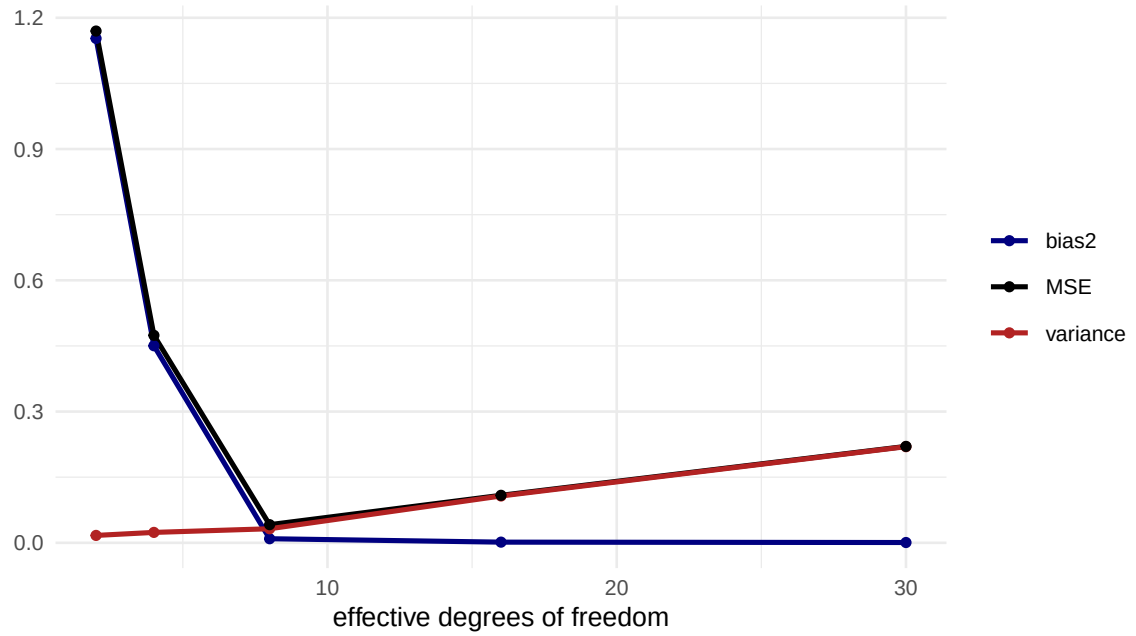

Figure 1: Squared bias falls and variance rises with flexibility; MSE is U-shaped. Every tuning parameter in this lab is this picture.

3 Regularized regression

A deliberately high-dimensional design matrix: all covariates, their squares, and all two-way interactions.

```
X <- model.matrix(~ (educ + exper + wordsum + pareduc + prestige + hours +
                     female + black + otherace + factor(year) + region)^2 +
                    I(exper^2) + I(educ^2) + I(hours^2) - 1, data = gss)
y <- gss$lnearn
dim(X)
#> [1] 3509  219
```

```
n <- nrow(X); train <- sample(n, floor(0.7 * n)); test <- setdiff(seq_len(n), train)
mse <- function(pred, truth) mean((pred - truth)^2)
```

```
## OLS on the full design -- note the warning about rank deficiency
ols <- lm(y[train] ~ X[train, ])
pred_ols <- cbind(1, X[test, ]) %*% ifelse(is.na(coef(ols)), 0, coef(ols))
c(train_mse = mse(fitted(ols), y[train]), test_mse = mse(pred_ols, y[test]))
#> train_mse  test_mse
#>    0.5494    0.6381
```

The gap between training and test error is the **optimism**. It is large here because the design is flexible relative to n.

3.1 Ridge and lasso

```
cv_ridge <- cv.glmnet(X[train, ], y[train], alpha = 0, nfolds = 10)
cv_lasso <- cv.glmnet(X[train, ], y[train], alpha = 1, nfolds = 10)
cv_enet  <- cv.glmnet(X[train, ], y[train], alpha = 0.5, nfolds = 10)

c(ridge_lambda_min = cv_ridge$lambda.min,
  lasso_lambda_min = cv_lasso$lambda.min,
  lasso_lambda_1se = cv_lasso$lambda.1se)
#> ridge_lambda_min lasso_lambda_min lasso_lambda_1se
#>          0.70359          0.02004          0.07371
```

```
plot(cv_lasso)
```

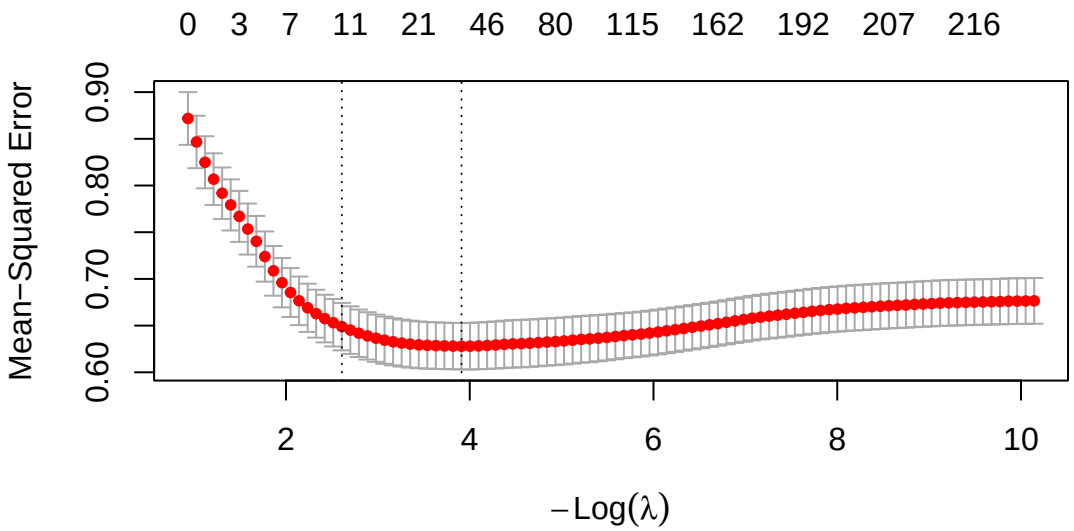

Figure 2: Cross-validated MSE against log penalty for the lasso. The left dashed line is the minimizer; the right is the one-standard-error rule.

```
preds <- list(
  ridge = predict(cv_ridge, X[test, ], s = "lambda.min"),
  lasso = predict(cv_lasso, X[test, ], s = "lambda.min"),
  lasso_1se = predict(cv_lasso, X[test, ], s = "lambda.1se"),
  enet  = predict(cv_enet,  X[test, ], s = "lambda.min")
)
sapply(preds, mse, truth = y[test]) |> round(5)
#>     ridge     lasso lasso_1se      enet
#>    0.6046    0.6018    0.6244    0.6001
```

```
c(p_total = ncol(X),
  nonzero_min = sum(coef(cv_lasso, s = "lambda.min") != 0) - 1,
  nonzero_1se = sum(coef(cv_lasso, s = "lambda.1se") != 0) - 1)
```

```
#>     p_total nonzero_min nonzero_1se
#>         219          36          11
```

The one-standard-error rule buys a much smaller model at almost no predictive cost. That is usually the right default when you have to describe the model.

3.2 Lasso selection is unstable

```
picked <- replicate(20, {
  b <- sample(train, length(train), replace = TRUE)
  cvb <- cv.glmnet(X[b, ], y[b], alpha = 1, nfolds = 5)
  nm <- rownames(coef(cvb, s = "lambda.min"))
  nm[as.vector(coef(cvb, s = "lambda.min")) != 0][-1]
}, simplify = FALSE)

tab <- sort(table(unlist(picked)), decreasing = TRUE)
head(tab, 12)
#>
#>                  black:factor(year)2016                           exper:prestige
#>                                      20                                       20
#>  factor(year)2012:regionMiddle Atlantic factor(year)2018:regionE. South Central
#>                                      20                                       20
#>                    female:regionPacific            black:regionE. South Central
#>                                      20                                       19
#>                             exper:hours   factor(year)2012:regionSouth Atlantic
#>                                      19                                       19
#>   factor(year)2018:regionSouth Atlantic                                I(exper^2)
#>                                      19                                       19
#>               otherace:factor(year)2018          otherace:regionSouth Atlantic
#>                                      19                                       19
c(variables_ever_selected = length(tab),
  selected_every_time = sum(tab == 20))
#> variables_ever_selected     selected_every_time
#>                     218                       5
```

> **!** Important
>
> Across 20 bootstrap resamples, only a handful of variables are chosen every time, while dozens are chosen occasionally. **The selected set is not a ranking of importance.** Among correlated predictors, lasso picks one essentially arbitrarily. Reporting "lasso selected these variables" as a substantive finding is a mistake — and reporting naive p-values from a refit on the selected set is worse.

4 Trees and ensembles

```
df_tr <- as.data.frame(gss[train, ])
df_te <- as.data.frame(gss[test, ])
form <- lnearn ~ educ + exper + wordsum + pareduc + prestige + hours +
  female + black + otherace + region + year

tree <- rpart(form, data = df_tr, control = rpart.control(cp = 0.002))
```

```
rpart.plot(tree, type = 2, extra = 0, cex = .55, box.palette = "Blues")
```

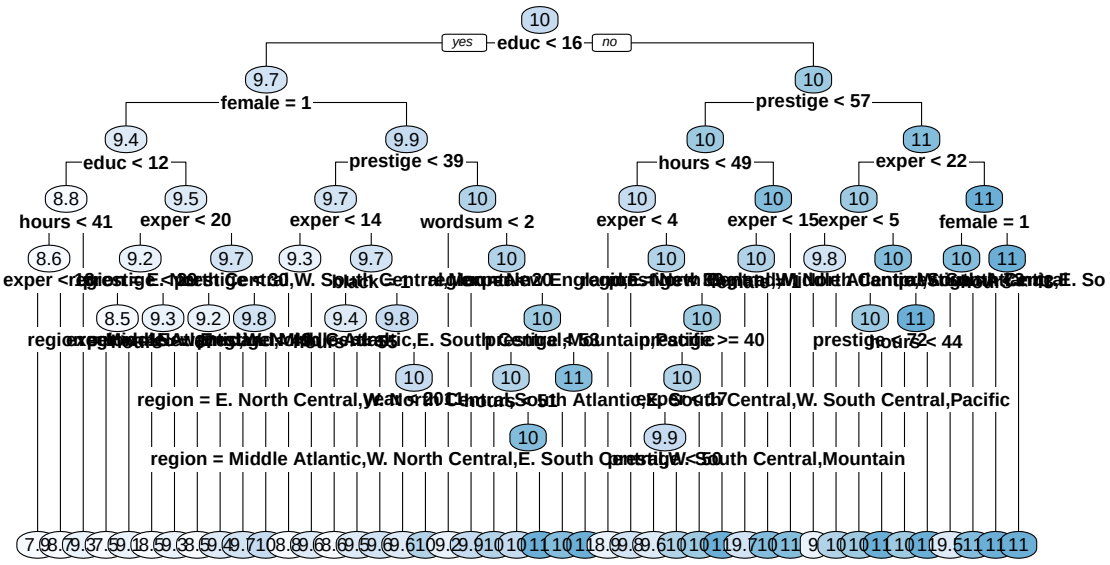

Figure 3: A regression tree grown with a complexity penalty. Readable, and high-variance.

```
rf <- ranger(form, data = df_tr, num.trees = 800, mtry = 4,
             min.node.size = 5, seed = 1)

## boosting, hand-rolled so the mechanism is visible
boost <- function(data, newdata, M = 400, eta = 0.05, depth = 3) {
  f_tr <- rep(mean(data$lnearn), nrow(data))
  f_te <- rep(mean(data$lnearn), nrow(newdata))
  for (m in seq_len(M)) {
    data$r <- data$lnearn - f_tr
    t_m <- rpart(update(form, r ~ .), data = data,
                 control = rpart.control(maxdepth = depth, cp = 0, xval = 0))
    f_tr <- f_tr + eta * predict(t_m, data)
    f_te <- f_te + eta * predict(t_m, newdata)
  }
  f_te
```

```
}
pred_boost <- boost(df_tr, df_te)

c(tree   = mse(predict(tree, df_te), y[test]),
  forest = mse(predict(rf, df_te)$predictions, y[test]),
  boost  = mse(pred_boost, y[test]),
  lasso  = mse(preds$lasso, y[test]),
  ridge  = mse(preds$ridge, y[test]))
#>   tree forest  boost  lasso  ridge
#> 0.6725 0.5812 0.5736 0.6018 0.6046
```

Note the ordering: a single tree is worst, the ensembles improve on it substantially, and on this modestly sized tabular problem the penalized linear models are competitive. **Flexibility is not automatically better.**

```
c(oob_mse = rf$prediction.error, test_mse = mse(predict(rf, df_te)$predictions, y[test]))
#>  oob_mse test_mse
#>   0.6203   0.5812
```

Out-of-bag error is a free honest estimate of test error — no separate split needed.

5 How to do cross-validation wrong

```
## WRONG: screen variables on the full sample, then cross-validate
cors <- apply(X, 2, function(z) abs(cor(z, y)))
top <- order(cors, decreasing = TRUE)[1:40]
cv_leak <- cv.glmnet(X[, top], y, alpha = 1, nfolds = 10)
leak_cv_mse <- min(cv_leak$cvm)

## RIGHT: screen inside each fold
folds <- sample(rep(1:10, length.out = n))
right <- sapply(1:10, function(k) {
  tr <- which(folds != k); te <- which(folds == k)
  cc <- apply(X[tr, ], 2, function(z) abs(cor(z, y[tr])))
  tp <- order(cc, decreasing = TRUE)[1:40]
  fit <- cv.glmnet(X[tr, tp], y[tr], alpha = 1, nfolds = 5)
  mse(predict(fit, X[te, tp], s = "lambda.min"), y[te])
})

c(leaky_cv_mse = leak_cv_mse, honest_cv_mse = mean(right))
#>  leaky_cv_mse honest_cv_mse
#>        0.6146        0.6159
```

> **Warning**
>
> The leaky procedure looks better because the screening step already saw the held-out outcomes. **Every data-dependent choice must happen inside the fold** — screening, standardization, imputation, and tuning alike.

6 Exercises

1. **Ridge versus lasso by design.** Simulate two datasets with $p = 200$: one sparse (5 non-zero coefficients), one dense (200 small coefficients). Which penalty wins in each, and why?
2. **Tuning a forest.** Vary `mtry` over $\{2, 4, 8, 12\}$ and `min.node.size` over $\{1, 5, 20\}$. Which matters more? Explain via bias and variance.
3. **Boosting's dials.** In `boost()`, try $\eta \in \{0.01, 0.1, 0.5\}$ with $M \in \{100, 400, 2000\}$. Show that small η with large M beats large η with small M, and explain why in terms of bias.
4. **Clustered splitting.** Re-run the cross-validation splitting on `psu` rather than on rows. Does the estimated test error change? Which is the honest number, and why does the Week 2 bootstrap rule apply here too?
5. **Prediction is not inference.** Take the lasso's `educ` coefficient at `lambda.min`. Compare it with the OLS estimate from Week 1. Explain in two sentences why you would not report the lasso coefficient with a standard error.

7 Session information

```
sessionInfo()
#> R version 4.4.1 (2024-06-14)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS 26.5.2
#>
#> Matrix products: default
#> BLAS:   /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;
#>
#> locale:
#> [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
#>
#> time zone: Asia/Shanghai
#> tzcode source: internal
#>
#> attached base packages:
```

```
#> [1] stats     graphics  grDevices utils     datasets  methods   base
#>
#> other attached packages:
#> [1] rpart.plot_3.1.3 rpart_4.1.23     ranger_0.17.0    glmnet_4.1-10
#> [5] Matrix_1.7-0     ggplot2_4.0.0    dplyr_1.1.4
#>
#> loaded via a namespace (and not attached):
#>  [1] gtable_0.3.6       jsonlite_1.8.9     compiler_4.4.1     tinytex_0.53
#>  [5] tidyselect_1.2.1   Rcpp_1.1.1         tidyr_1.3.1        splines_4.4.1
#>  [9] scales_1.4.0       yaml_2.3.10        fastmap_1.2.0      lattice_0.22-7
#> [13] R6_2.5.1           labeling_0.4.3     generics_0.1.3     shape_1.4.6.1
#> [17] knitr_1.48         iterators_1.0.14   tibble_3.2.1       pillar_1.9.0
#> [21] RColorBrewer_1.1-3 rlang_1.3.0        utf8_1.2.4         xfun_0.47
#> [25] S7_0.2.0           cli_3.6.5          withr_3.0.1        magrittr_2.0.3
#> [29] digest_0.6.37      foreach_1.5.2      grid_4.4.1         lifecycle_1.0.4
#> [33] vctrs_0.6.5        evaluate_1.0.0     glue_1.7.0         farver_2.1.2
#> [37] codetools_0.2-20   survival_3.6-4     fansi_1.0.6        purrr_1.0.2
#> [41] rmarkdown_2.28     tools_4.4.1        pkgconfig_2.0.3    htmltools_0.5.8.1
```